

PROJECT IMPLEMENTATION REPORT

Three-Tier E-Commerce Application on AWS with Auto Scaling, Application Load Balancer (ALB), and Amazon RDS

A Step-by-Step Documentation of the Cloud Deployment Process

Submitted by: Leegan S

Course: AWS Cloud Computing

Batch: July [11.30-1.00]

1. Project Overview

1.1 Project Title

Three-Tier E-Commerce Application on AWS with Auto Scaling, Application Load Balancer (ALB), and Amazon RDS.

1.2 Project Description

This project implements a production-style e-commerce web application deployed on Amazon Web Services (AWS) using a three-tier architecture. The application allows users to browse products, register and log in, add items to a shopping cart, and complete checkout. The backend is built using Node.js and Express.js, exposing REST APIs consumed by an HTML/CSS/JavaScript frontend styled with Bootstrap. Application data — including product catalog, user accounts, cart items, and orders — is persisted in an Amazon RDS MySQL database. The compute tier runs on Amazon EC2 instances that are packaged into a custom Amazon Machine Image (AMI) and deployed through a Launch Template, an Auto Scaling Group, and an Application Load Balancer to achieve high availability, fault tolerance, and horizontal scalability.

1.3 Project Objectives

- Deploy a relational database layer using Amazon RDS (MySQL) with a properly configured security group and Multi-AZ/availability settings.
- Provision and configure an EC2 instance to host the Node.js/Express application and connect it to the RDS database.
- Package the fully configured server into a custom AMI to standardize and automate future instance provisioning.
- Create a Launch Template based on the custom AMI to define instance configuration for scaling.
- Configure a Target Group and Application Load Balancer to distribute incoming traffic across multiple instances.
- Configure an Auto Scaling Group to automatically launch or terminate instances based on demand, ensuring high availability.
- Develop and integrate a functional e-commerce frontend and backend with full CRUD operations against the RDS database.
- Test the end-to-end system, including load balancing, auto scaling, and application functionality, and document all issues and resolutions.

1.4 Three-Tier Architecture Overview

The application follows a three-tier architecture consisting of a presentation tier, an application tier, and a data tier. The presentation tier is the HTML/CSS/JavaScript frontend served to the browser. The application tier is the Node.js/Express backend running on EC2 instances behind an Application Load Balancer and managed by an Auto Scaling Group. The data tier is the Amazon RDS MySQL database, which is isolated in its own security group and accessed only by the application tier. Traffic flows from the client

to the ALB, which forwards requests to healthy EC2 instances registered in the Target Group; each instance processes the request and communicates with RDS for data persistence.

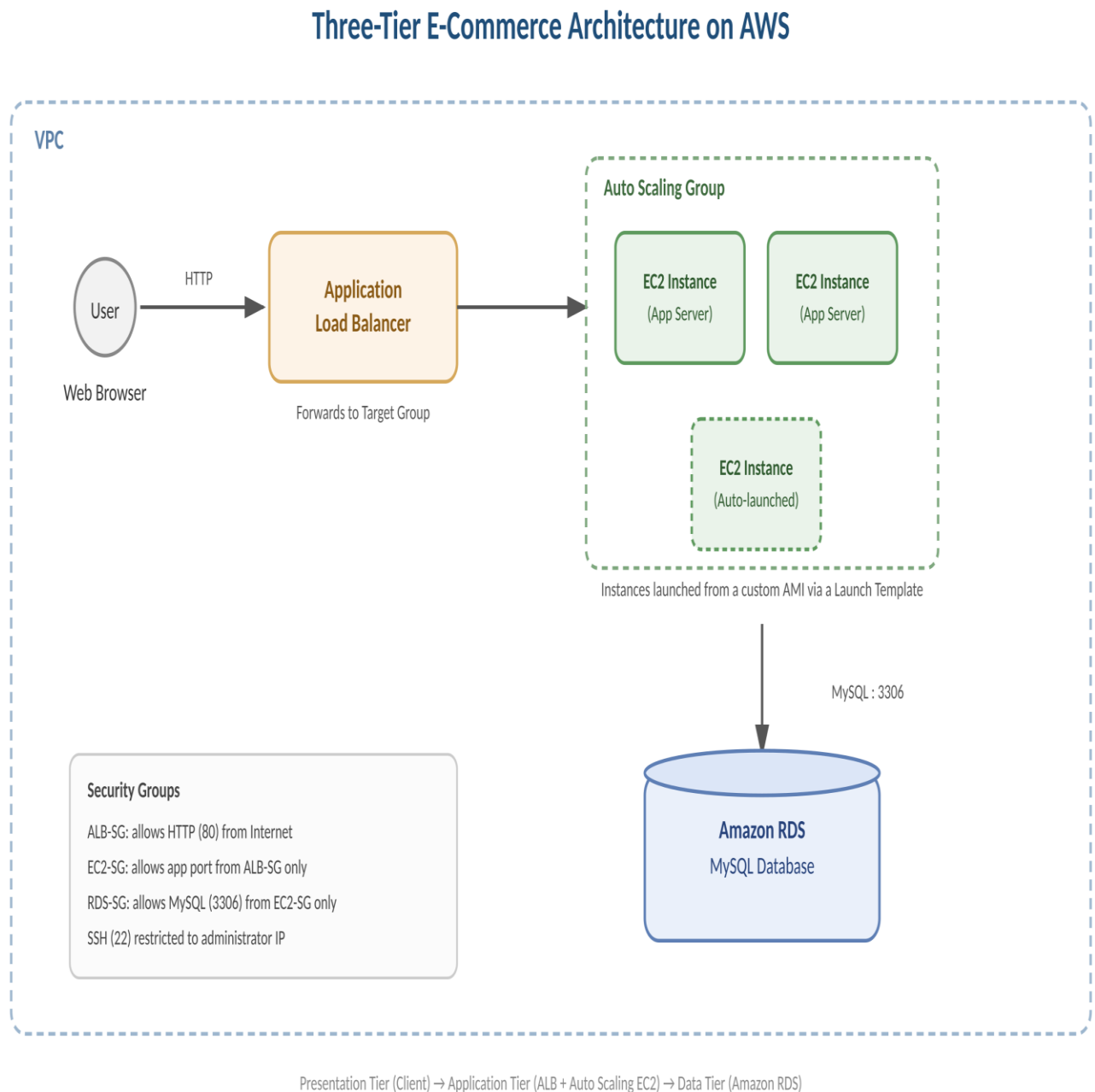


Figure 1: High-level three-tier architecture showing the ALB, Auto Scaling Group of EC2 instances, and RDS database.

2. Amazon RDS Creation

The first component implemented was the data tier. An Amazon RDS instance running MySQL was created to serve as the central, managed database for the application, removing the need to manually install and maintain a database server.

2.1 Creating the Amazon RDS MySQL Database

1. Opened the AWS Management Console and navigated to the RDS service.
2. Selected "Create database" and chose the "Standard create" method.
3. Selected MySQL as the database engine and chose an appropriate engine version.
4. Selected the Free Tier / appropriately sized template based on project requirements.
5. Set the DB instance identifier, master username, and master password.

2.2 Database Configuration

- DB instance identifier assigned to uniquely identify the database instance.
- Master username and a strong master password configured for administrative access.
- DB instance class selected based on the expected workload (burstable class for development/testing).

2.3 Storage Configuration

- Storage type set to General Purpose SSD (gp3) for a balance of cost and performance.
- Allocated storage set according to expected data volume for products, users, and orders.
- Storage autoscaling enabled to automatically increase capacity if the database approaches its allocated limit.

2.4 Security Group Configuration

A dedicated VPC security group was configured for the RDS instance to restrict access to only the application tier.

- Created/selected a security group scoped to the database.
- Added an inbound rule permitting TCP traffic on port 3306 (MySQL) from the EC2 security group only, rather than from the public internet.
- Left all other inbound traffic blocked to minimize the attack surface on the data tier.

2.5 Availability Settings

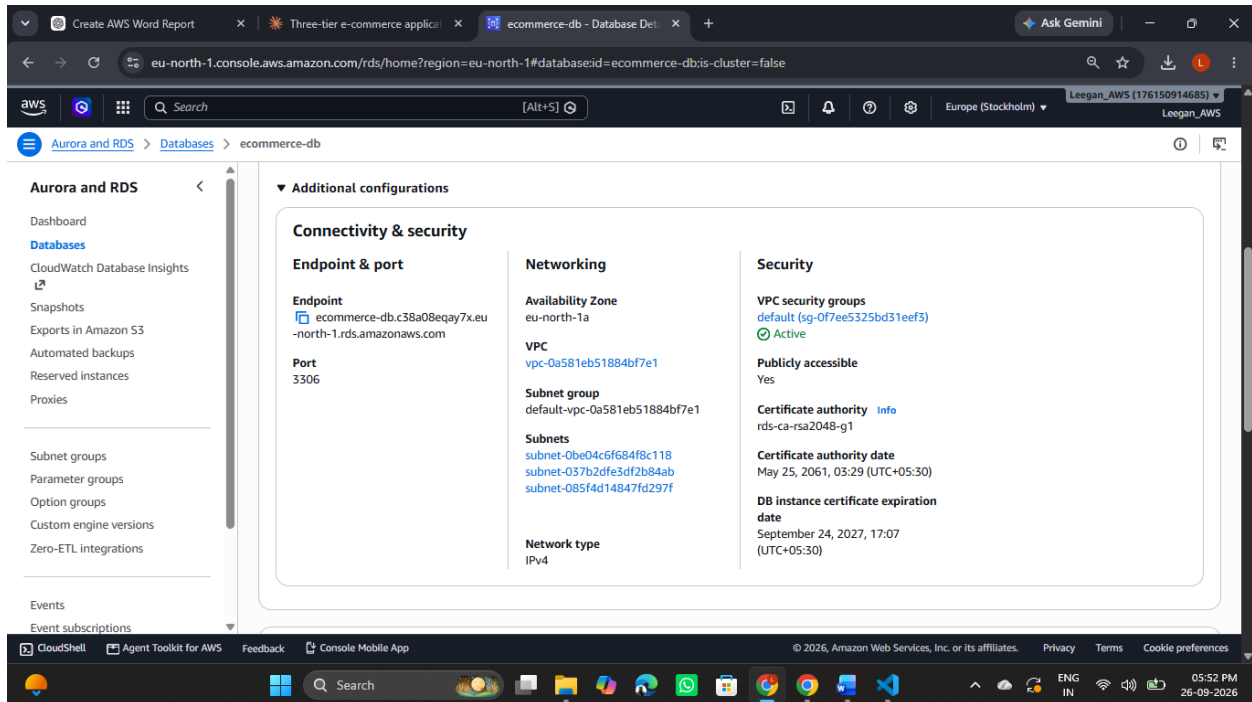
- Reviewed Multi-AZ deployment options for high availability and automatic failover.
- Selected the appropriate availability configuration for the project's requirements.
- Confirmed the VPC and subnet group placed the database across the intended Availability Zones.

2.6 Database Endpoint Generation

Once the database instance finished provisioning, AWS automatically generated a unique connection endpoint (hostname) and port for the instance. This endpoint was copied and used in all subsequent steps to connect the database from MySQL Workbench and from the application's environment configuration.

2.7 Verifying RDS Status

- Monitored the database instance status in the RDS console until it changed from "Creating" to "Available."
- Confirmed the endpoint, port, and security group were correctly attached under the Connectivity & Security tab



3. Connecting Amazon RDS to MySQL Workbench

3.1 Installing MySQL Workbench

MySQL Workbench was downloaded and installed on the local machine to provide a graphical interface for managing the RDS database, running SQL scripts, and verifying data without needing to connect through the command line.

3.2 Creating a New Connection

1. Opened MySQL Workbench and clicked the "+" icon next to MySQL Connections.
2. Entered a connection name to identify the RDS database.
3. Set the connection method to "Standard (TCP/IP)".

3.3 Using the RDS Endpoint

The RDS endpoint generated in Section 2.6 was entered as the Hostname for the new connection, allowing Workbench to route all queries directly to the managed database instance.

3.4 Configuring Port, Username, and Password

- Port set to 3306, matching the port opened in the RDS security group.
- Username set to the master username created during RDS setup.
- Password stored securely in Workbench's credential vault.

3.5 Testing the Database Connection

The "Test Connection" button was used to confirm that Workbench could successfully reach the RDS instance over the network and authenticate using the configured credentials.

3.6 Creating the Database

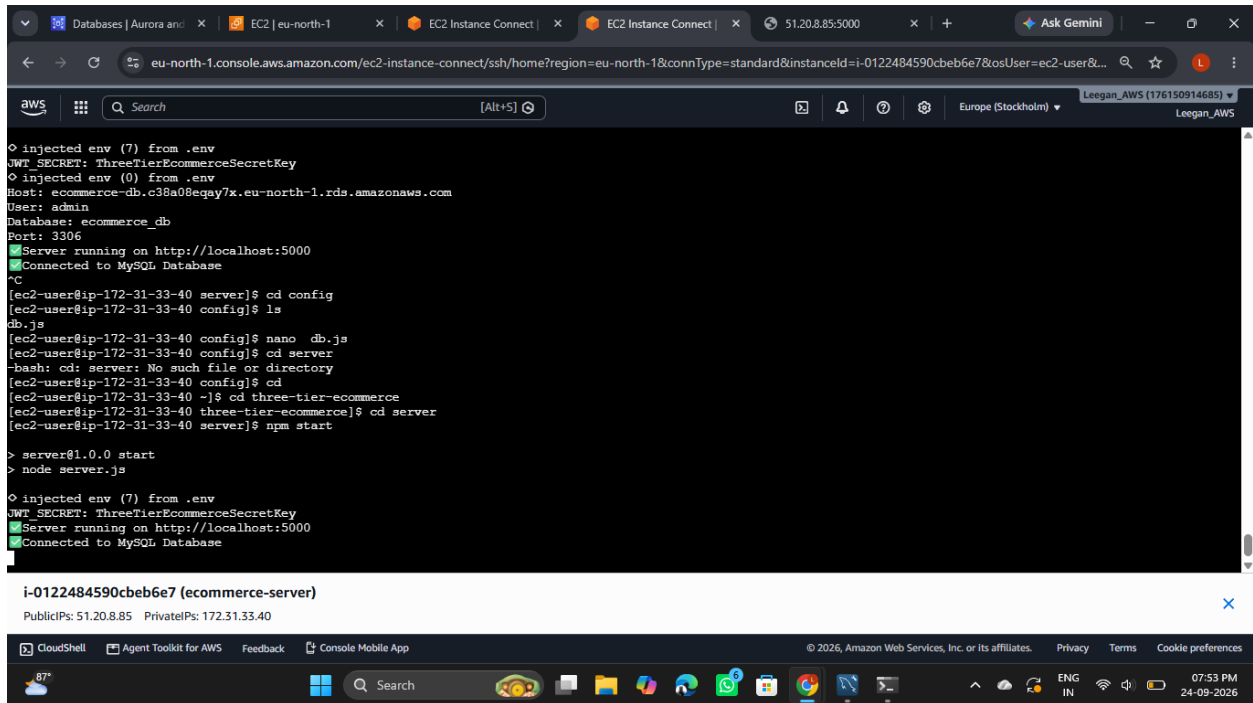
A new schema named "ecommerce_db" was created inside the RDS instance using MySQL Workbench's schema creation dialog, and set as the active schema for all subsequent table creation.

3.7 Creating/Importing Tables

Core tables required by the application were created inside ecommerce_db to support user accounts, the product catalog, shopping cart entries, and orders — namely users, products, orders, and order_items, with foreign keys linking orders to users and order_items to both orders and products.

3.8 Verifying Successful Connectivity

- Ran a sample SELECT query against each table to confirm they were created correctly and were queryable.
- Confirmed the schema was visible under the RDS connection in the Workbench Navigator panel.



```
◇ injected env (7) from .env
JWT_SECRET: ThreeTierEcommerceSecretKey
◇ injected env (0) from .env
Host: ecommerce-db.c38a08eqay7x.eu-north-1.rds.amazonaws.com
User: admin
Database: ecommerce_db
Port: 3306
✓ Server running on http://localhost:5000
✓ Connected to MySQL Database
^C
[ec2-user@ip-172-31-33-40 server]$ cd config
[ec2-user@ip-172-31-33-40 config]$ ls
db.js
[ec2-user@ip-172-31-33-40 config]$ nano db.js
[ec2-user@ip-172-31-33-40 config]$ cd server
-bash: cd: server: No such file or directory
[ec2-user@ip-172-31-33-40 config]$ cd
[ec2-user@ip-172-31-33-40 ~]$ cd three-tier-ecommerce
[ec2-user@ip-172-31-33-40 three-tier-ecommerce]$ cd server
[ec2-user@ip-172-31-33-40 server]$ npm start

> server@1.0.0 start
> node server.js

◇ injected env (7) from .env
JWT_SECRET: ThreeTierEcommerceSecretKey
✓ Server running on http://localhost:5000
✓ Connected to MySQL Database
```

i-0122484590cbeb6e7 (ecommerce-server)

PublicIPs: 51.20.8.85 PrivateIPs: 172.31.33.40

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

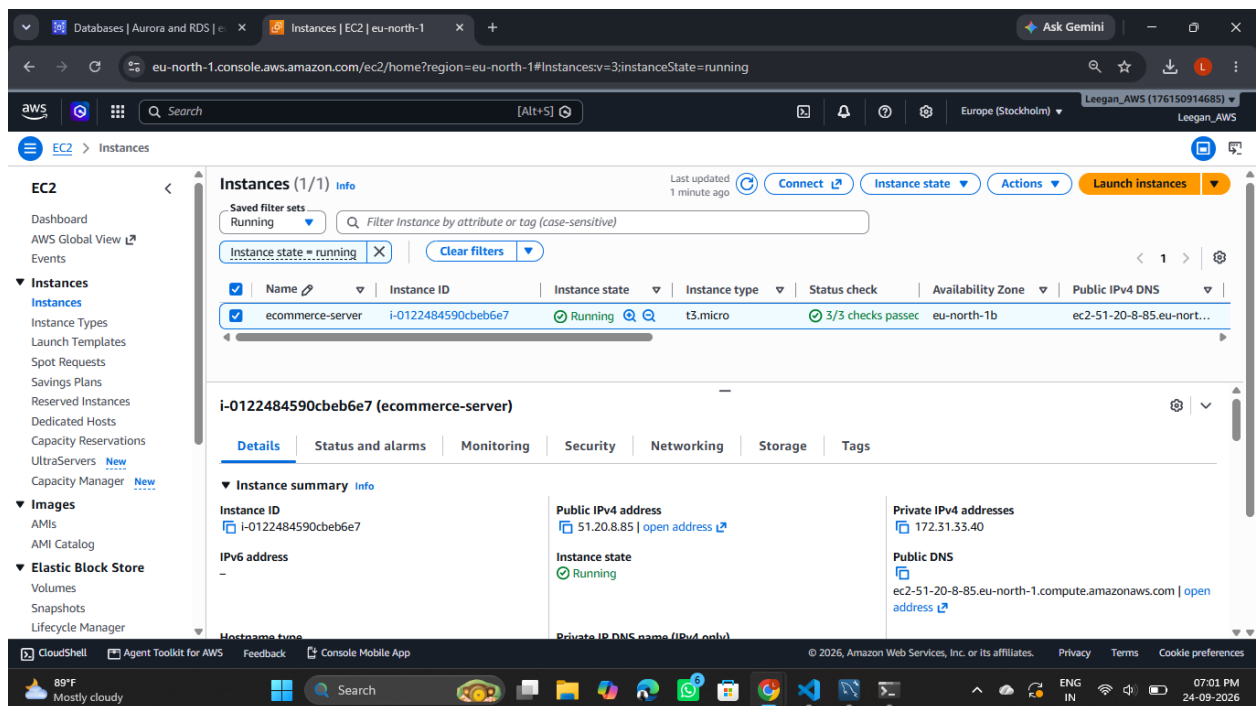
87° Search ENG IN 07:53 PM 24-09-2026

4. EC2 Instance Creation

4.1 Launching an Amazon Linux EC2 Instance

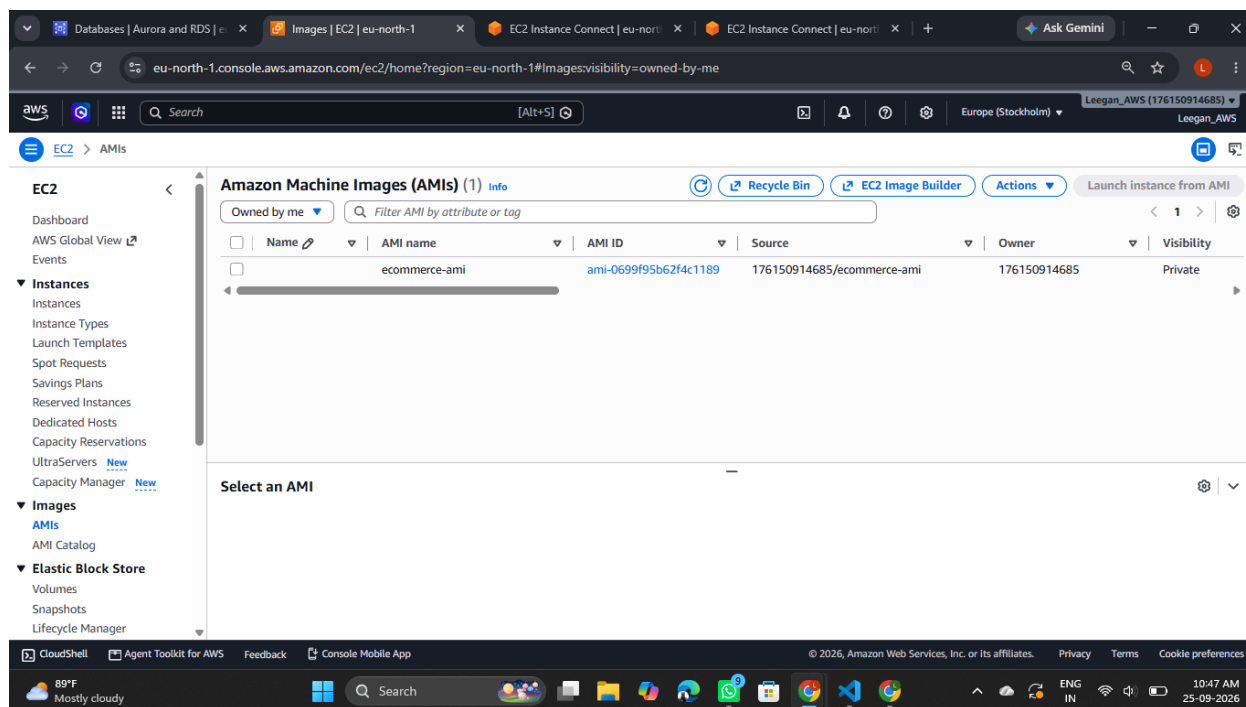
With the database ready, an EC2 instance was launched to host the Node.js/Express backend and serve the frontend assets.

1. Navigated to the EC2 console and selected "Launch instance."
2. Provided a descriptive name for the instance.



4.2 Selecting the AMI

Amazon Linux 2023 was selected as the base Amazon Machine Image (AMI) for its lightweight footprint, long-term support, and native compatibility with AWS tooling.



4.3 Choosing the Instance Type

A t2.micro / t3.micro instance type was selected, providing sufficient compute for the Node.js application during development and testing while remaining cost-efficient.

4.4 Configuring the Key Pair

- Created a new key pair and downloaded the .pem private key file.
- Stored the key file securely and restricted its file permissions for SSH access.

4.5 Configuring the Security Group

A security group was created for the EC2 instance to control inbound traffic:

- Allowed SSH (port 22) from the administrator's IP address for secure remote access.
- Allowed HTTP (port 80) and the application port (e.g., 3000) from the ALB security group / public internet as required for web traffic.
- All other ports left closed by default.

4.6 Connecting to the Instance Using SSH

```
chmod 400 ecommerce.pem
ssh -i "ecommerce.pem" ec2-user@<EC2-13.53.236.133>
```

5. Linux Server Configuration

After connecting to the instance, the server environment was prepared to run the Node.js application and connect to the RDS database.

5.1 Updating the Linux Server

```
sudo yum update -y
```

5.2 Installing Git

```
sudo yum install git -y  
git --version
```

5.3 Installing Node.js

```
sudo yum install -y nodejs  
node -v
```

5.4 Installing npm

npm is bundled with the Node.js installation above; its version was verified as follows:

```
npm -v
```

5.5 Installing PM2

PM2 was installed globally to keep the Node.js application running continuously, restart it automatically on crashes, and allow it to start automatically on server reboot.

```
sudo npm install -g pm2
```

5.6 Cloning the Project from GitHub

```
git clone https://github.com/<your-username>/<your-repo>.git  
cd three-tier-ecommerce
```

5.7 Installing Project Dependencies

```
npm install
```

5.8 Configuring Environment Variables

A .env file was created on the server to store database connection details and application configuration outside of the source code.

```
DB_HOST=ecommerce-db.c38a08eqay7x.eu-north-1.rds.amazonaws.com  
DB_USER=admin
```

```
DB_PASSWORD=Admin321  
DB_NAME=ecommerce_db  
PORT=3000
```

5.9 Connecting the Backend to Amazon RDS

The backend's database connection module was configured to read the environment variables above and establish a connection pool to the RDS MySQL instance using the mysql2 driver.

5.10 Starting the Application

```
pm2 start server.js --name ecommerce-app  
pm2 save  
pm2 startup
```

5.11 Verifying Deployment

- Accessed the application using the EC2 instance's public IP address and configured port in a browser.
- Confirmed the homepage loaded correctly and the backend responded to API requests.
- Checked application logs using pm2 logs to confirm there were no startup errors.

6. Amazon Machine Image (AMI) Creation

6.1 Creating an AMI from the Configured EC2 Instance

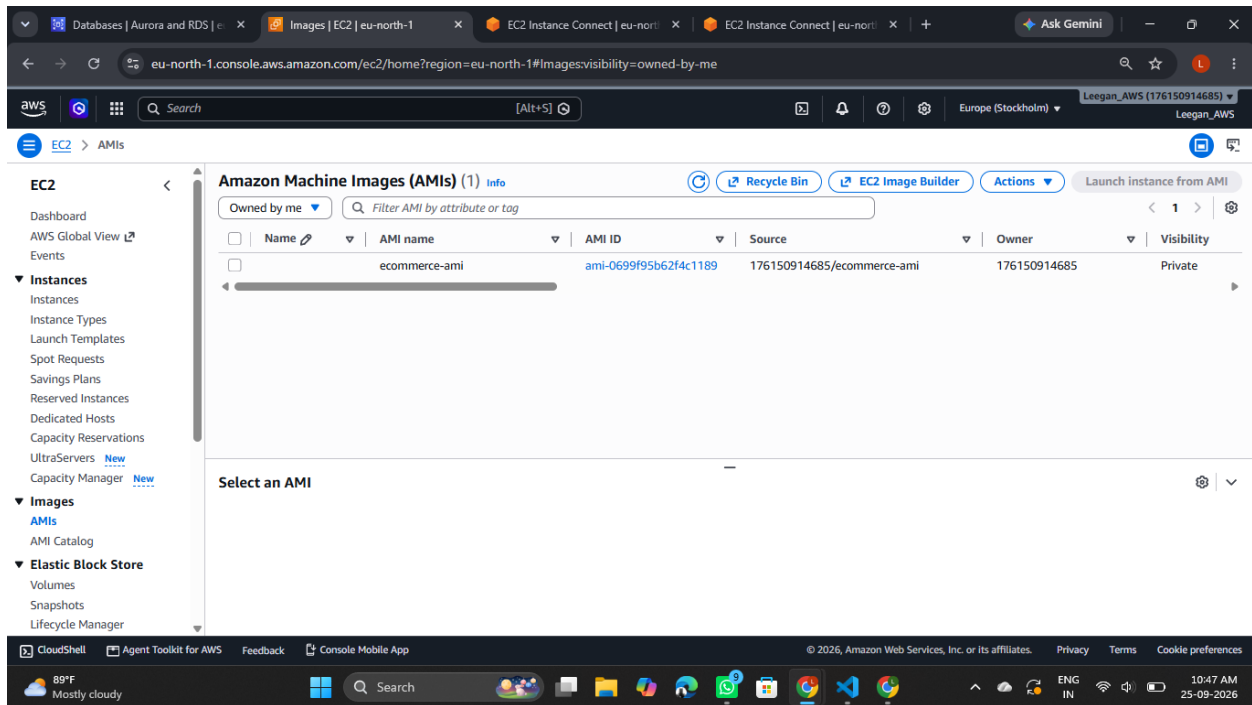
1. Selected the fully configured EC2 instance in the console.
2. Chose Actions → Image and templates → Create image.
3. Provided a name and description for the image and confirmed creation.

6.2 Purpose of Creating the AMI

Creating a custom AMI captured the exact state of the server — including Node.js, npm, PM2, application code, and configuration — into a reusable template. This AMI became the foundation for the Launch Template, ensuring that every new instance launched by the Auto Scaling Group would be identically configured and immediately ready to serve traffic, without any manual setup.

6.3 Verifying the AMI

- Navigated to EC2 → AMIs and confirmed the new AMI status changed to "Available."
- Launched a temporary test instance from the AMI to confirm the application started correctly without additional configuration.



7. Launch Template Creation

7.1 Creating the Launch Template

A Launch Template was created to standardize how the Auto Scaling Group would provision new EC2 instances.

7.2 Selecting the Custom AMI

The custom AMI created in Section 6 was selected as the base image, ensuring every new instance launched with Node.js, PM2, and the application already installed and running.

7.3 Configuring the Instance Type

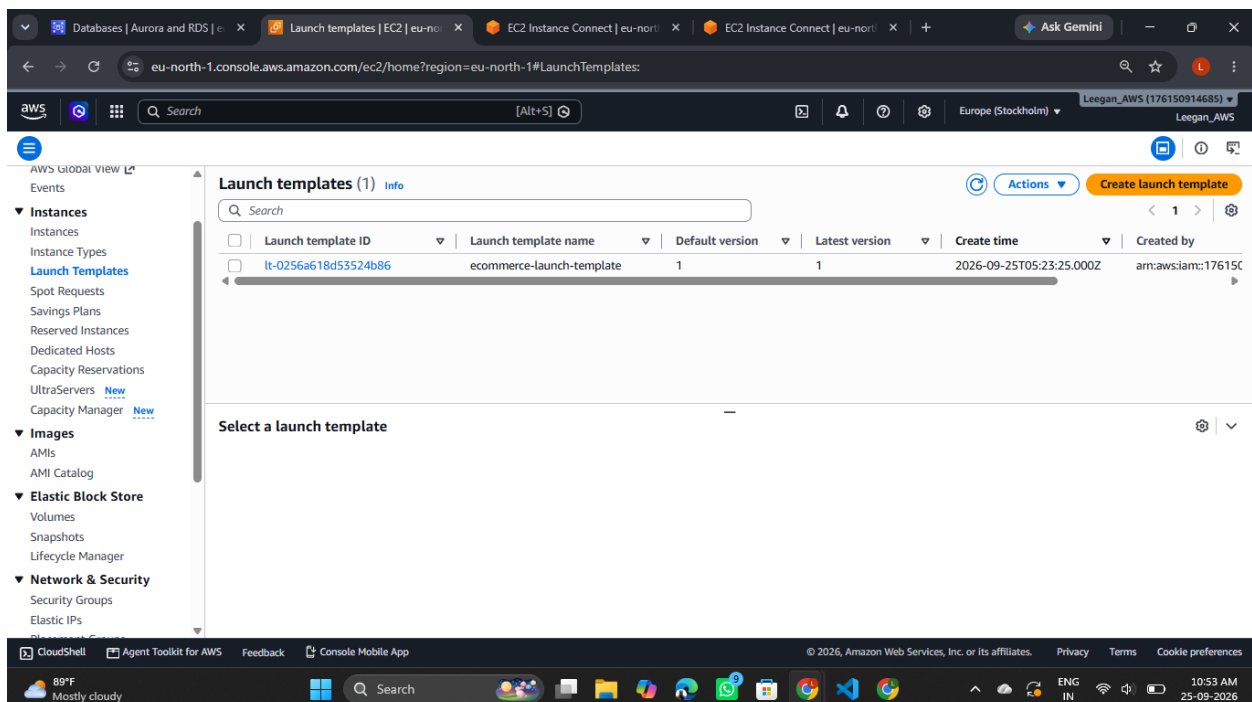
The same instance type used during development (t2.micro / t3.micro) was selected to keep behavior consistent between the original instance and instances launched by Auto Scaling.

7.4 Configuring the Security Group

The EC2 security group configured in Section 4.5 was attached to the Launch Template so that every new instance inherited the correct inbound rules for HTTP/application traffic and SSH access.

7.5 Configuring Storage

The root EBS volume size and type were configured to match the original instance's storage, ensuring sufficient space for the application and logs.



8. Target Group Creation

8.1 Creating the Target Group

1. Navigated to EC2 → Target Groups → Create target group.
2. Selected "Instances" as the target type.
3. Set the protocol and port matching the application (e.g., HTTP on the app's listening port).
4. Selected the same VPC used by the EC2 instance and RDS database.

8.2 Registering EC2 Instances

The originally configured EC2 instance was registered as an initial target to verify the target group before attaching it to the Auto Scaling Group.

8.3 Configuring Health Checks

- Health check protocol set to HTTP.
- Health check path set to the application's root or a dedicated health endpoint (e.g., "/").
- Healthy/unhealthy threshold counts and timeout intervals configured to detect failures promptly without flapping.

8.4 Verifying Healthy Targets

After registration, the target group's Targets tab was monitored until the instance's health status changed to "healthy," confirming the application was reachable and responding correctly on the configured port.

eu-north-1.console.aws.amazon.com/ec2/home?region=eu-north-1#TargetGroup:targetGroupArn=arn:aws:elasticloadbalancing:eu-north-1:176150914685:targetgroup/...

Successfully created the target group: **ecommerce-target-group**. Anomaly detection is automatically applied to all registered targets. Results can be viewed in the Targets tab.

ecommerce-target-group

Details

arn:aws:elasticloadbalancing:eu-north-1:176150914685:targetgroup/ecommerce-target-group/81a0d1be4445d74

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-0a581eb51884bf7e1
IP address type IPv4	Load balancer None associated		

1 Total targets	0 Healthy 0 Anomalous	0 Unhealthy	1 Unused	0 Initial	0 Draining
--------------------	-----------------------------	----------------	-------------	--------------	---------------

► **Distribution of targets by Availability Zone (AZ)**
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

9. Application Load Balancer (ALB)

9.1 Creating the ALB

1. Navigated to EC2 → Load Balancers → Create load balancer and selected "Application Load Balancer."
 2. Provided a name and selected "Internet-facing" as the scheme.
 3. Selected the VPC and at least two subnets across different Availability Zones for high availability.
- to accept incoming client requests and forward them to the Target Group created in Section 8.

9.2 Associating the Target Group

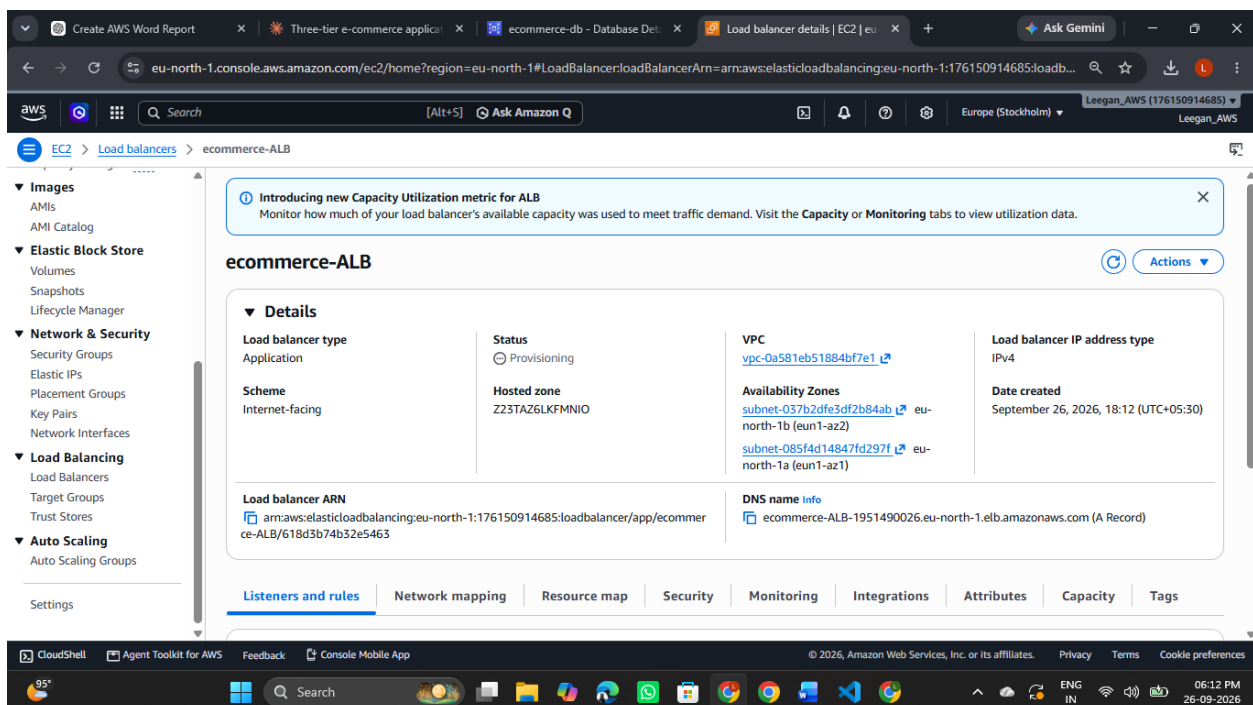
The listener's default action was set to forward traffic to the Target Group, linking the ALB to the registered EC2 instance(s).

9.3 Security Group Configuration

A security group was attached to the ALB allowing inbound HTTP (port 80) traffic from the internet, while the EC2 security group was updated to accept traffic only from the ALB's security group rather than from the public internet directly.

9.4 Testing Request Forwarding

The ALB's DNS name was accessed in a browser to confirm that requests were being forwarded correctly to the backend EC2 instance and that the application responded as expected.



10. Auto Scaling Group

10.1 Creating the Auto Scaling Group

An Auto Scaling Group (ASG) was created to automatically manage the number of running EC2 instances based on demand and health status.

10.2 Selecting the Launch Template

The Launch Template created in Section 7 was selected as the source configuration for all instances launched by the ASG.

10.3 Selecting Subnets

The same subnets used for the ALB were selected to ensure new instances were distributed across multiple Availability Zones.

10.4 Attaching the Target Group

The existing Target Group was attached to the ASG, so every instance launched or terminated by the ASG would automatically be registered with, or deregistered from, the ALB.

10.5 Configuring Minimum, Desired, and Maximum Capacity

- Minimum capacity set to ensure at least one instance is always running.
- Desired capacity set to the normal expected number of instances under regular load.
- Maximum capacity set to cap the number of instances that can be launched during high demand, controlling cost.

10.6 Health Checks

ELB health checks were enabled on the ASG, so that instances failing the ALB's health check would be automatically terminated and replaced with a new, healthy instance launched from the Launch Template.

The screenshot displays the AWS Management Console interface for Auto Scaling groups. At the top, the 'Auto Scaling groups (1/1)' section shows a table with one group, 'e-commerce-asg', which is 'At desired capacity' and '1/1 Healthy'. Below this, the 'Auto Scaling group: e-commerce-asg' details are shown, including a 'Capacity overview' card that indicates a 'Desired capacity' of 1 and 'Scaling limits' of 1-2. The console also shows various navigation tabs like 'Details', 'Integrations', and 'Automatic scaling'.

10.7 Testing Auto Scaling

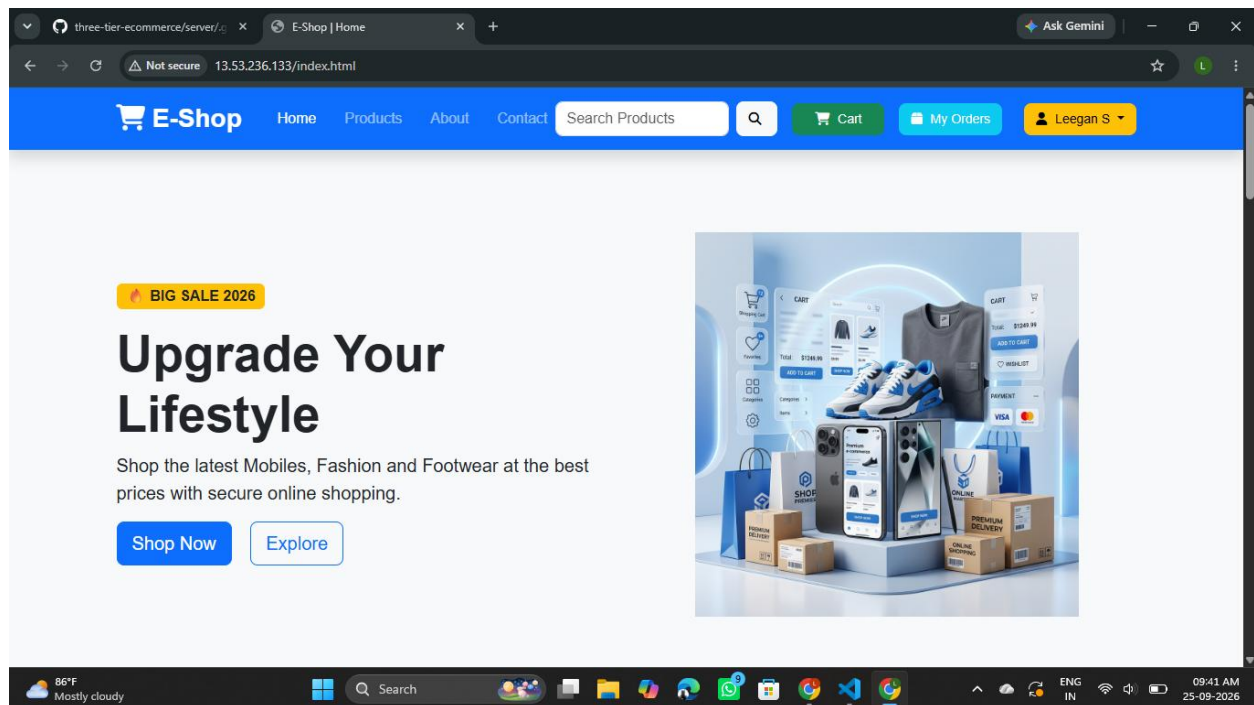
- Manually terminated an instance to confirm the ASG automatically launched a replacement instance from the Launch Template.
- Verified the replacement instance registered itself with the Target Group and passed health checks without manual intervention.
- Observed scaling activity in the ASG's Activity tab to confirm expected scale-out/scale-in behavior.

11. Frontend Development

The frontend was built using HTML, CSS, JavaScript, and the Bootstrap framework to provide a responsive, user-friendly shopping experience.

11.1 Home Page

The home page displays featured products in a responsive grid, pulling product data from the backend's products API and rendering it dynamically with JavaScript.

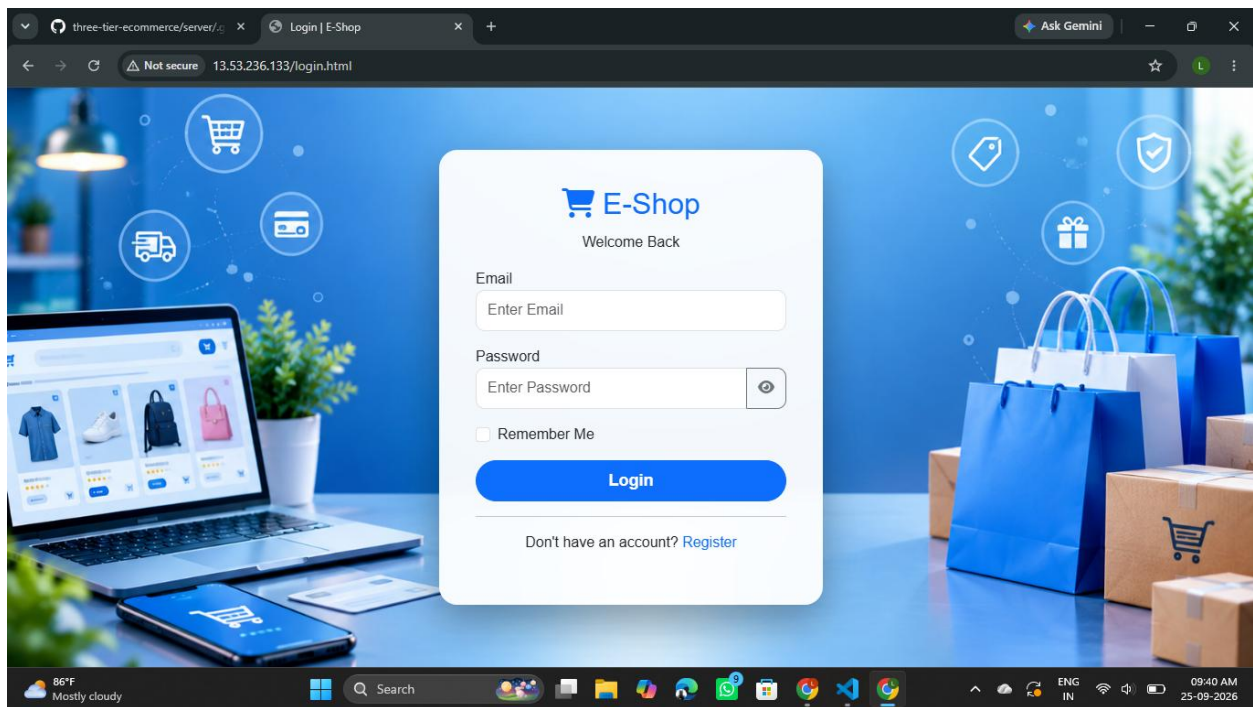


11.2 Login

A login form collects the user's email and password and submits them to the authentication API; on success, a session/token is stored client-side and the user is redirected to the home page.

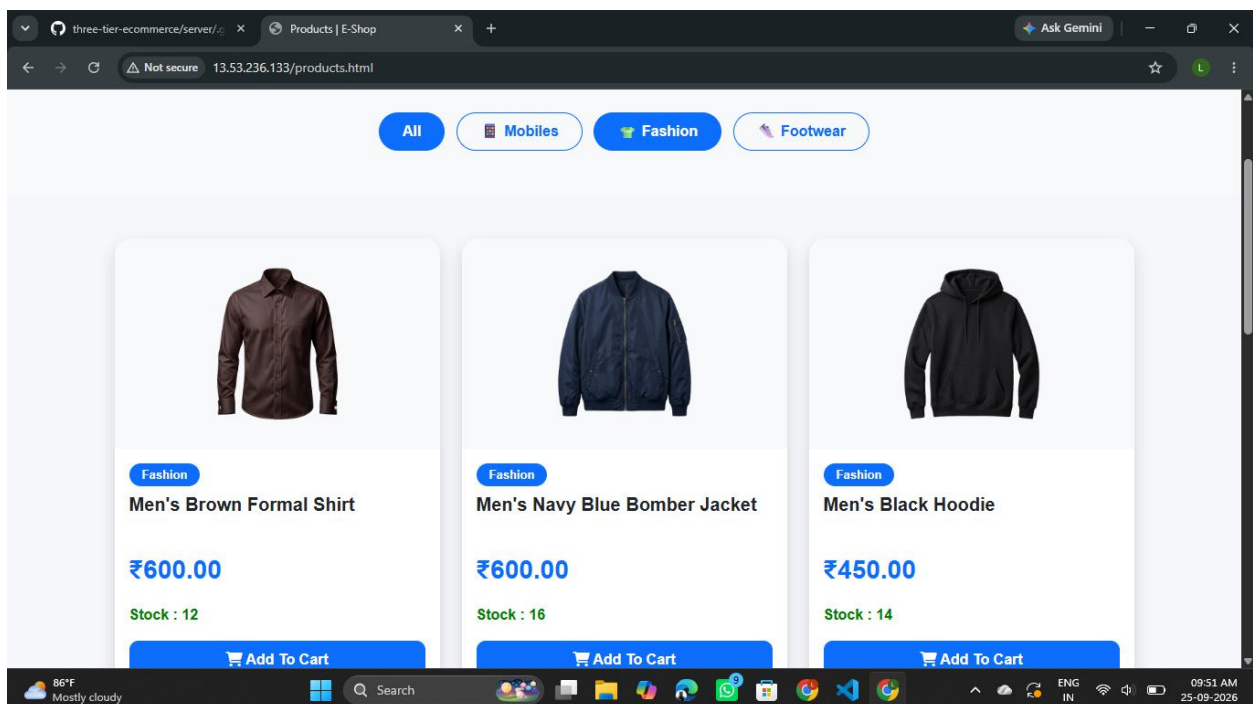
11.3 Registration

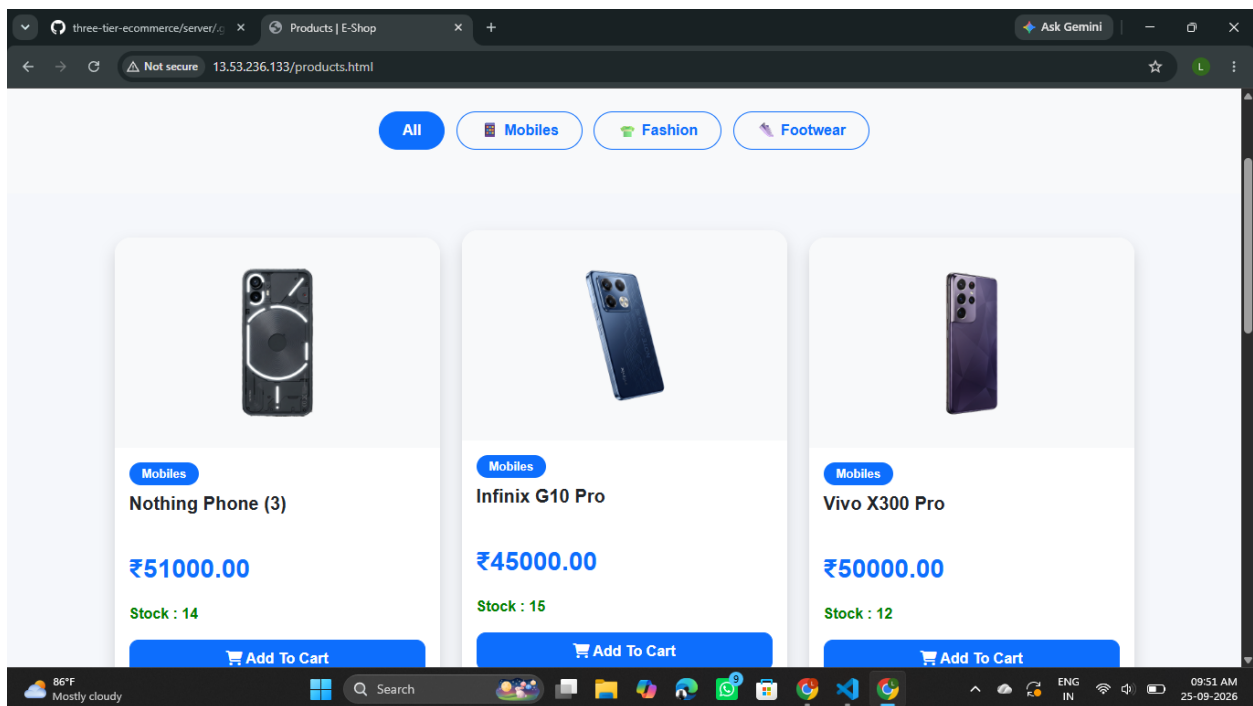
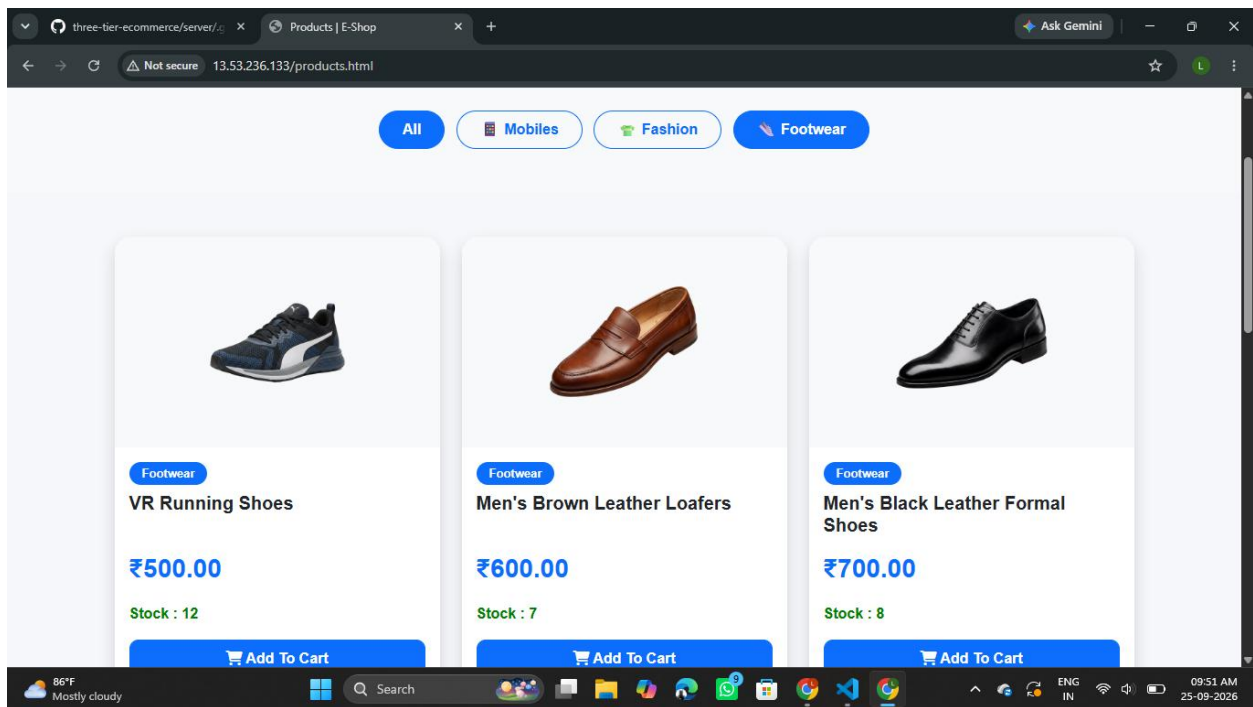
A registration form allows new users to create an account by submitting name, email, and password, which are validated and stored via the users API.



11.4 Product Listing

The product listing page fetches all available products from the backend and renders them as Bootstrap cards, each showing the product image, name, and price.





11.5 Product Details

Clicking a product opens a details view showing the full description, price, and stock availability, along with an "Add to Cart" action.

11.6 Shopping Cart

The cart page lists all items the user has added, allowing quantity updates and item removal, and calculates the running subtotal dynamically.

11.7 Checkout

The checkout page collects any final confirmation details and submits the order to the backend, which creates the order and order-item records in the database.

11.8 Responsive Design

Bootstrap's grid system and responsive utility classes were used throughout the frontend so the layout adapts correctly across desktop, tablet, and mobile screen sizes.

11.9 Navigation Bar

A fixed Bootstrap navigation bar provides links to the home page, product listing, cart, and account actions (login/register/logout), remaining accessible from every page.

11.10 Footer

A footer section was added to each page containing basic site information and links, giving the application a complete, polished appearance.

12. Backend Development

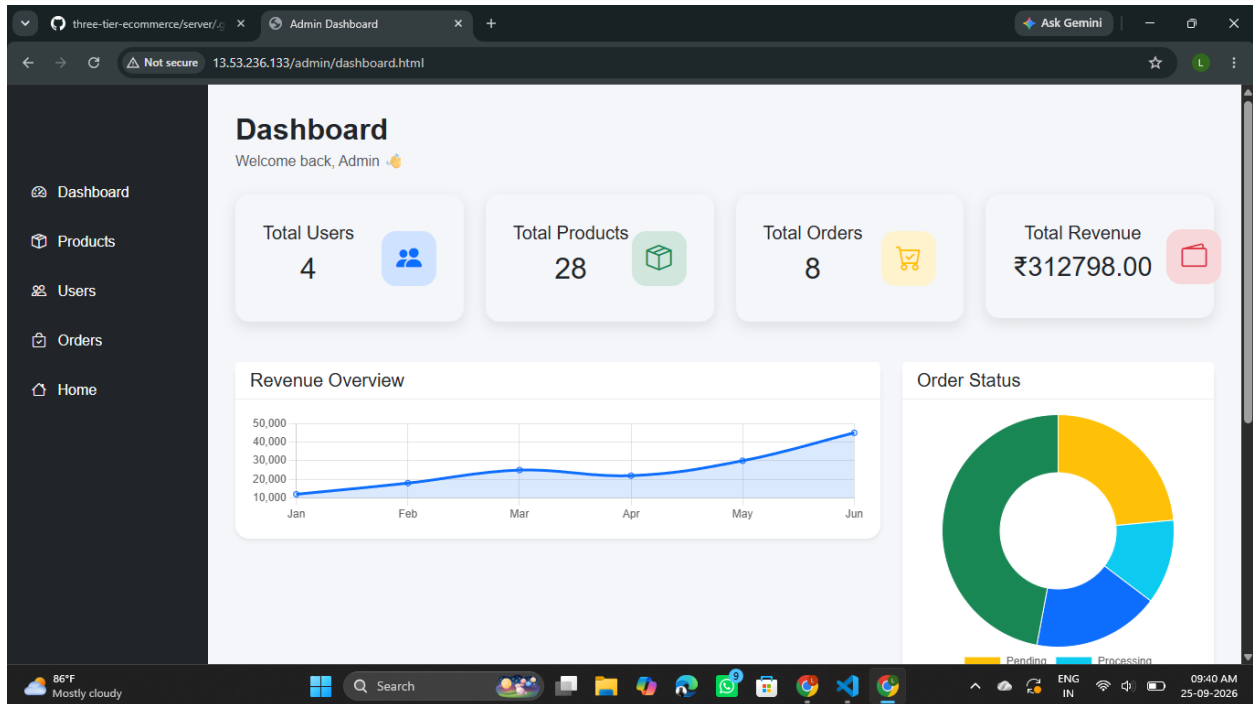
The backend was implemented using Node.js and the Express.js framework, exposing a set of REST APIs consumed by the frontend.

12.1 REST APIs

Express routes were organized by resource, exposing endpoints for authentication, products, cart, and orders, each returning JSON responses.

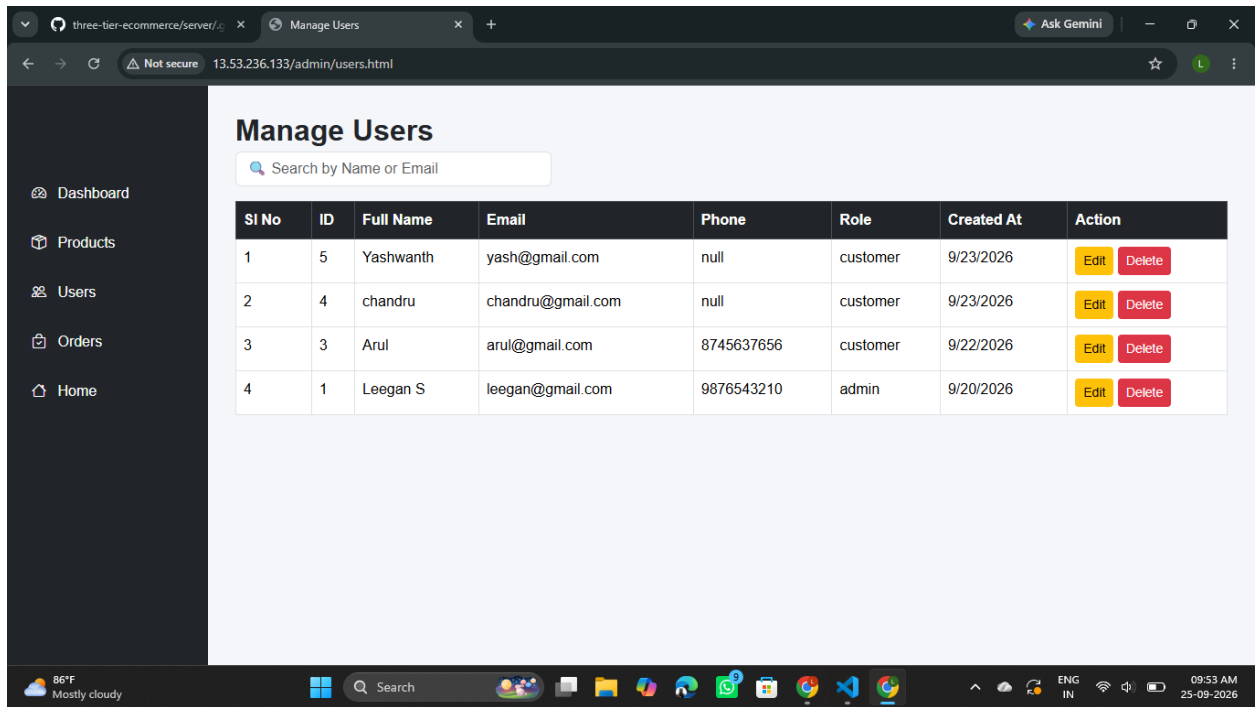
```
app.use('/api/auth', authRoutes);
app.use('/api/products', productRoutes);
app.use('/api/cart', cartRoutes);
app.use('/api/orders', orderRoutes);
```

12.2 Admin Dashboard



12.3 User Authentication

Registration and login endpoints were implemented with password hashing to avoid storing plaintext passwords, and a session/token mechanism to identify authenticated requests.



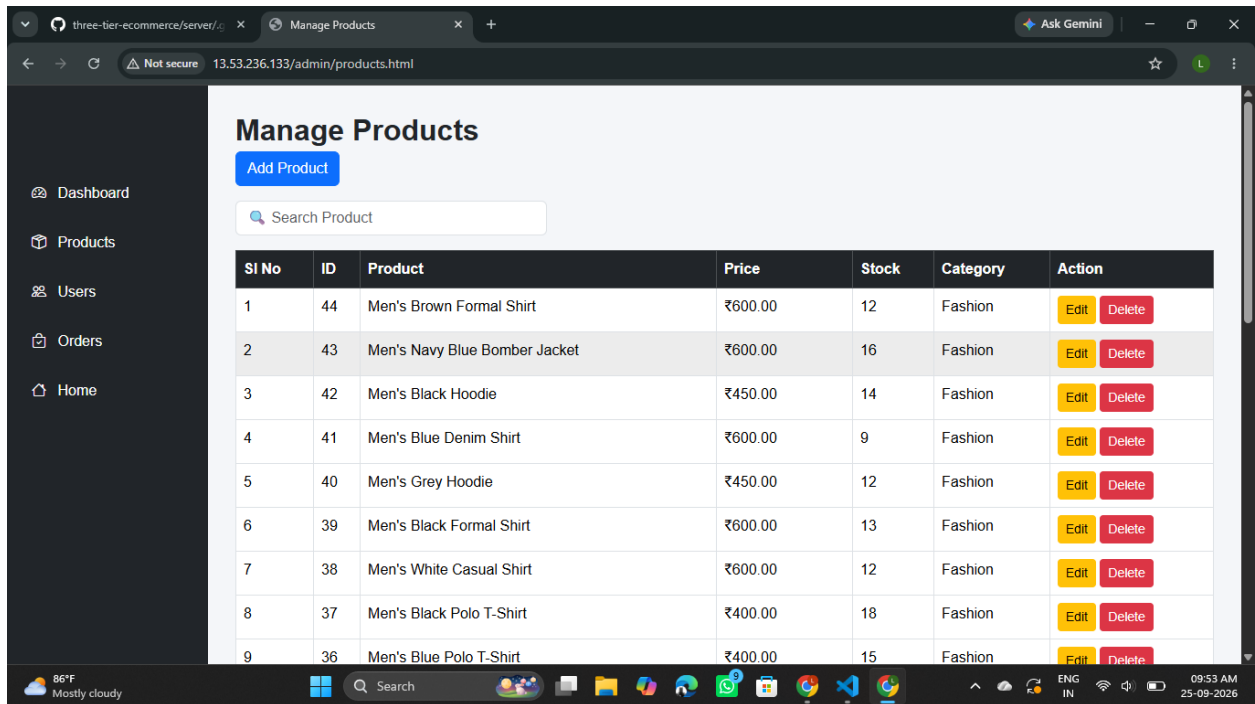
The screenshot shows a web browser window with the URL `13.53.236.133/admin/users.html`. The page title is "Manage Users". There is a search bar labeled "Search by Name or Email". Below the search bar is a table with the following data:

SI No	ID	Full Name	Email	Phone	Role	Created At	Action
1	5	Yashwanth	yash@gmail.com	null	customer	9/23/2026	Edit Delete
2	4	chandru	chandru@gmail.com	null	customer	9/23/2026	Edit Delete
3	3	Arul	arul@gmail.com	8745637656	customer	9/22/2026	Edit Delete
4	1	Leegan S	leegan@gmail.com	9876543210	admin	9/20/2026	Edit Delete

The sidebar on the left contains the following links: Dashboard, Products, Users, Orders, and Home. The browser's taskbar at the bottom shows the date and time as 09:53 AM on 25-09-2026.

12.4 Product Management

Endpoints were implemented to list all products, fetch a single product by ID, and (for administrative use) create, update, or remove products.



The screenshot shows a web browser window with the URL `13.53.236.133/admin/products.html`. The page title is "Manage Products". There is a search bar labeled "Search Product" and an "Add Product" button. Below the search bar is a table with the following data:

SI No	ID	Product	Price	Stock	Category	Action
1	44	Men's Brown Formal Shirt	₹600.00	12	Fashion	Edit Delete
2	43	Men's Navy Blue Bomber Jacket	₹600.00	16	Fashion	Edit Delete
3	42	Men's Black Hoodie	₹450.00	14	Fashion	Edit Delete
4	41	Men's Blue Denim Shirt	₹600.00	9	Fashion	Edit Delete
5	40	Men's Grey Hoodie	₹450.00	12	Fashion	Edit Delete
6	39	Men's Black Formal Shirt	₹600.00	13	Fashion	Edit Delete
7	38	Men's White Casual Shirt	₹600.00	12	Fashion	Edit Delete
8	37	Men's Black Polo T-Shirt	₹400.00	18	Fashion	Edit Delete
9	36	Men's Blue Polo T-Shirt	₹400.00	15	Fashion	Edit Delete

The sidebar on the left contains the following links: Dashboard, Products, Users, Orders, and Home. The browser's taskbar at the bottom shows the date and time as 09:53 AM on 25-09-2026.

12.5 Cart Management

Cart endpoints allow adding items, updating item quantities, removing items, and retrieving the current cart contents for a user session.

12.6 Order Management

The checkout endpoint validates the cart, creates an order record and associated order-item records, and clears the cart on success.

SI No	ID	Customer	Total Amount	Status	Date	Action
1	11	Leegan S	₹600.00	Pending	9/25/2026	View Update Status
2	10	Arul	₹400.00	Pending	9/23/2026	View Update Status
3	9	chandru	₹400.00	Pending	9/23/2026	View Update Status
4	8	Arul	₹400.00	Pending	9/23/2026	View Update Status
5	7	Yashwanth	₹400.00	Pending	9/23/2026	View Update Status
6	6	chandru	₹400.00	Pending	9/23/2026	View Update Status
7	5	Leegan S	₹800.00	Shipped	9/22/2026	View Update Status
8	4	Arul	₹50000.00	Delivered	9/22/2026	View Update Status
9	1	Leegan S	₹259998.00	Delivered	9/20/2026	View Update Status

12.7 Database Connectivity

The mysql2 driver was used to create a connection pool to the RDS instance using the credentials configured in the .env file, allowing efficient, reusable database connections across requests.

```
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10
});
```

13. Database Integration

13.1 Connecting the Backend Application with Amazon RDS

The backend's connection pool (Section 12.6) was pointed at the RDS endpoint from Section 2.6, and the application was restarted to confirm a successful connection on startup.

13.2 CRUD Operations

- Create: new users, cart items, and orders inserted via parameterized INSERT statements.
- Read: product listings, cart contents, and order history retrieved via SELECT queries.
- Update: cart item quantities and order statuses updated via UPDATE statements.
- Delete: cart items removed via DELETE statements when a user removes a product from their cart.

13.3 Database Testing

Each CRUD operation was exercised through the application UI while monitoring the corresponding table in MySQL Workbench to confirm that records were correctly inserted, updated, or removed.

13.4 Successful Integration

End-to-end integration was confirmed once user registration, product browsing, cart updates, and order placement all correctly persisted data to, and retrieved data from, the RDS database with no connection errors.

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'ecommerce_db' database with tables like 'cart', 'categories', 'order_items', 'orders', 'products', and 'users'. The 'users' table is selected. The central 'SQL Editor' pane contains the query: `show tables;` followed by `select * from users;`. The 'Result Grid' on the right displays the data from the 'users' table. The table has 7 rows of data, including an admin user and several customers. The bottom status bar indicates 'Query Completed'.

id	full_name	email	password	phone	address	role	created_at
1	Leegan S	leegan@gmail.com	\$2b\$10\$LevusdACHvQPE6dMcBIV.20a8b2RFq...	9876543210	Combatore	admin	2026-09-20 04:57:20
3	Arul	arul@gmail.com	\$2b\$10\$JotVidB7h9d0tzztVLn.evuOke1gh7...	8745637656	Chennai	customer	2026-09-22 03:53:08
4	chandru	chandru@gmail.com	\$2b\$10\$63/YM2i/q9Qe4fOeLu37OHeUoCTC/...	N/A	N/A	customer	2026-09-23 05:21:37
5	Yashwanth	yash@gmail.com	\$2b\$10\$61/9QZocP9PeGEn9AHOMv.r7h8tQ/j9J...	N/A	N/A	customer	2026-09-23 07:11:59
6	Prem Kumar	premkumar@gmail.com	\$2b\$10\$y2rQe3hvk3dzoH.R76qOp.ZFF1r6qaw...	N/A	N/A	customer	2026-09-26 06:02:56
7	Arun	Arun12@gmail.com	\$2b\$10\$ZLWsa7uXq3Adg.1mTQnA2F06f0tUJUL...	N/A	N/A	customer	2026-09-26 06:32:53
*	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Source Code:

Due to the large size of the project, the complete source code is maintained in a GitHub repository.

GitHub Repository:

<https://github.com/Leegan8484/three-tier-ecommerce>

14. Conclusion

This project successfully implemented a complete three-tier e-commerce application on AWS, covering database provisioning with Amazon RDS, application hosting on EC2, image standardization through a custom AMI, and automated, highly available deployment using a Launch Template, Target Group, Application Load Balancer, and Auto Scaling Group. The frontend and backend were built and integrated end-to-end, with all core e-commerce functionality — registration, login, product browsing, cart management, and checkout — tested and verified against the live RDS database. The issues encountered during deployment, particularly around security group rules, health checks, and AMI/User Data configuration, were identified and resolved, resulting in a stable, self-healing, and horizontally scalable deployment.